

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 11: SDS Part 1

Instructors: Justin Yokota

Agenda

- Overview
- Transistors
 - Building basic logic gates
 - Building Circuits
- Simplifying Circuits
 - Boolean Logic

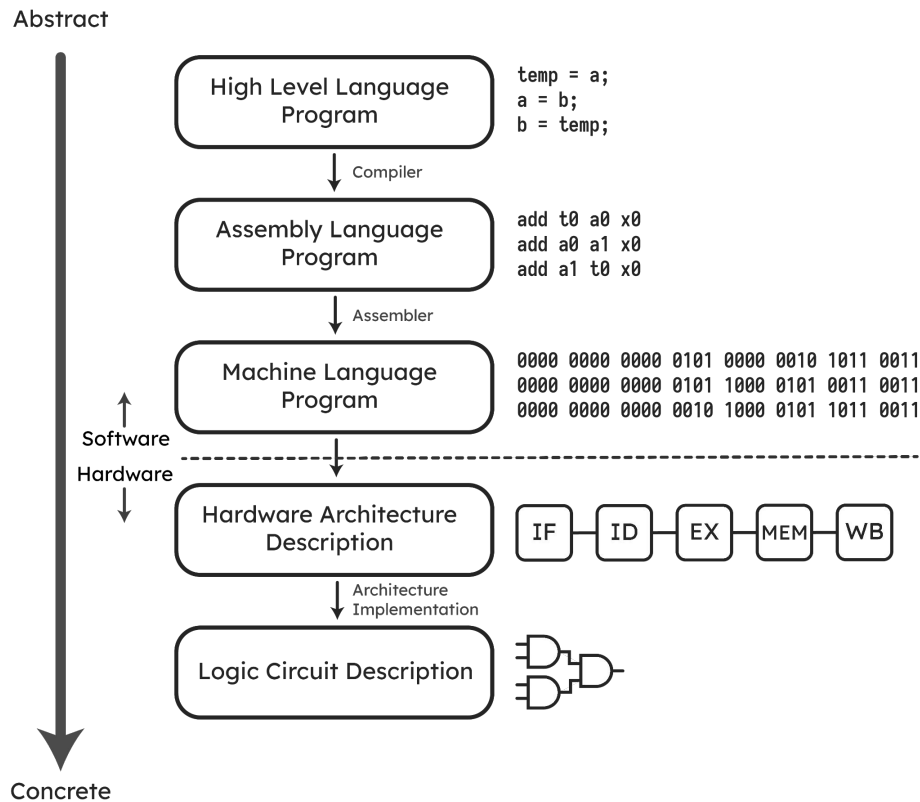
Agenda

- Overview
- Transistors
 - Building basic logic gates
 - Building Circuits
- Simplifying Circuits
 - Boolean Logic

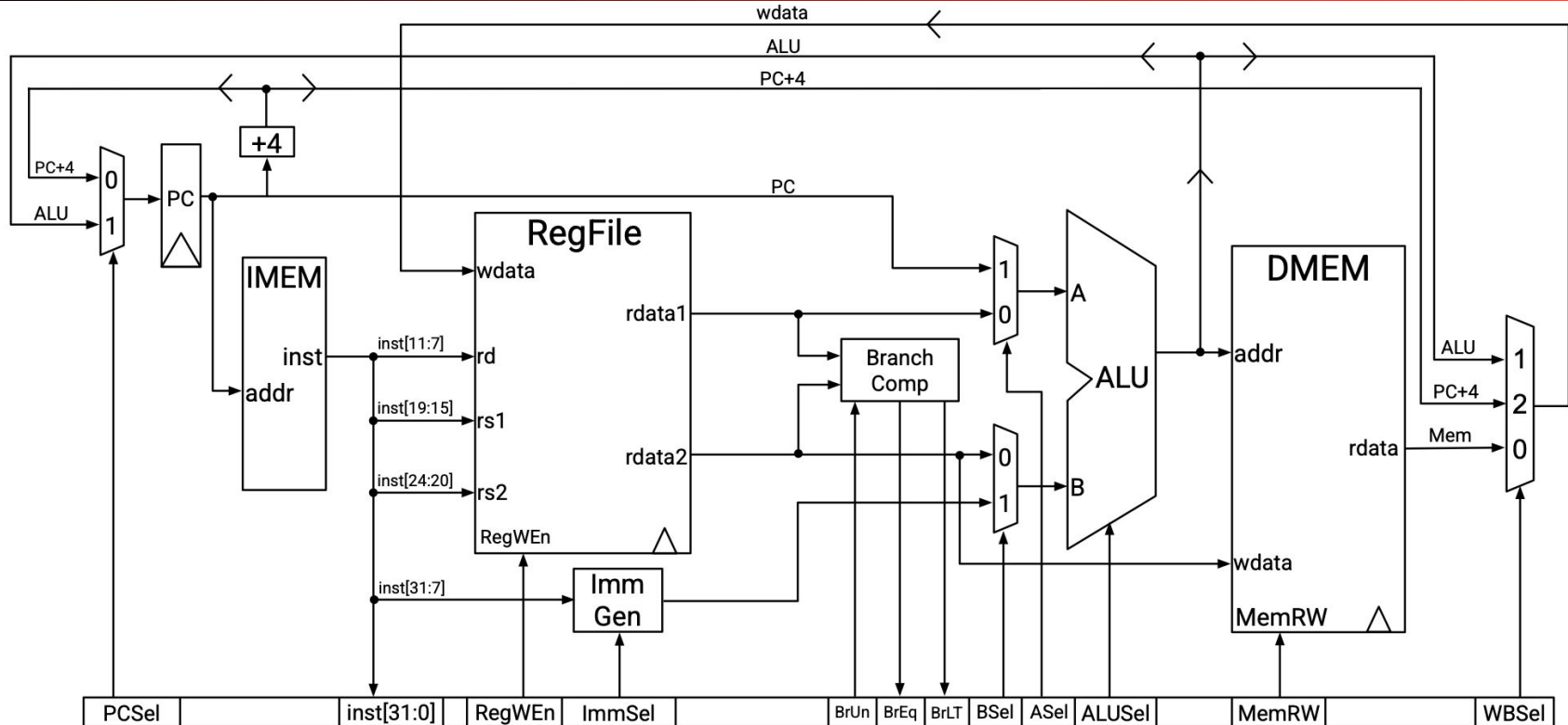
Hardware

- Over the next unit, we'll study how a modern processor is built, starting with the basic building blocks.
 - Why study hardware design? Go beyond “just programming.”
 - To really understand how computers work, we need to understand the complete stack, including the physical level.
 - Understand capabilities and limitations of HW in general, and processors in particular.
 - Why is my computer so slow? Why does my battery run down?
 - There is only so much you can do with standard processors!
 - For extra performance, you may need to design your own custom HW.
 - Prepare for more in depth HW courses (EECS 151, CS 152).
 - Get a Job: Apple is a traditional HW company. Even traditional SW companies (Google, Amazon, Meta) do their own hardware design!

Abstraction Map



Goal: Build this CPU from scratch



Synchronous Digital Systems (SDS)

- The hardware underlying almost every processor is a **Synchronous Digital System**.
 - **Synchronous**: All operations coordinated by a central clock.
 - “Heartbeat” of the system!
- **Digital**: Represent all values by discrete values, i.e., binary digits.
 - We’ve seen how to represent many symbols via 1s and 0s.
This representation extends to electrical signals!

Synchronous Digital Systems (SDS)

Two main parts:

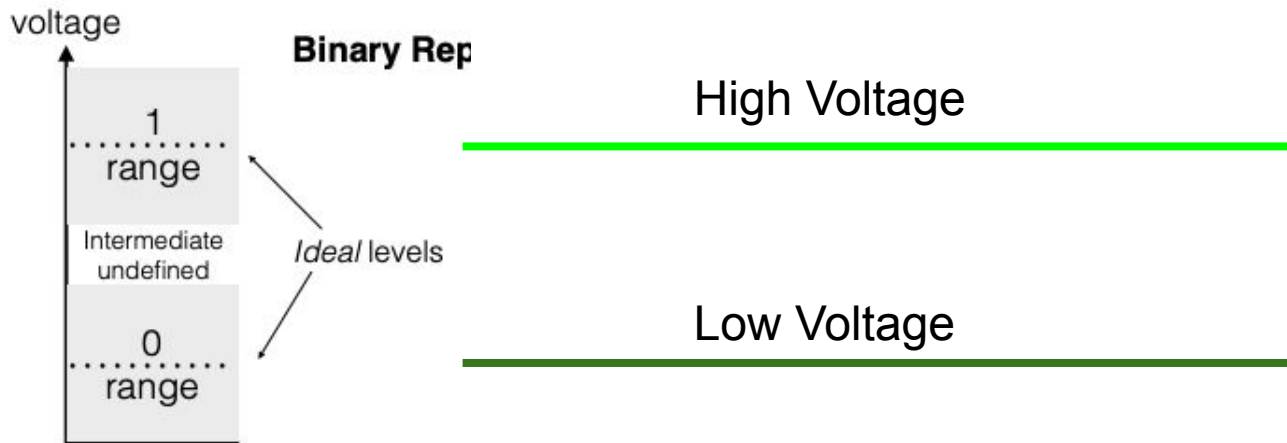
- Combinational Logic (today)
 - Computes functions in terms of inputs.
 - No way to store information, no side effects
- State Elements (next week)
 - Components that store information
 - Example: Registers

Agenda

- Overview
- **Transistors**
 - Building basic logic gates
 - Building Circuits
- Simplifying Circuits
 - Boolean Logic

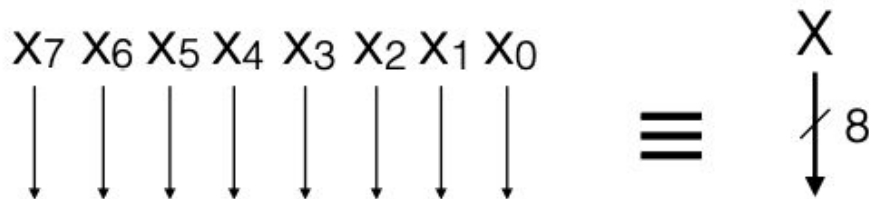
Wires

- On a chip/PC board, wires (i.e., electrical nodes) provide electrical signals and are used to represent variables.
 - A wire can take on different values at different points in time.
- **Binary signal representation.** Use voltage levels to signal 0 or 1.



Wires

- Multiple wires used to represent multiple bits
 - Why not multiple bits on same wire? Too much added complexity; simpler can be made more efficient

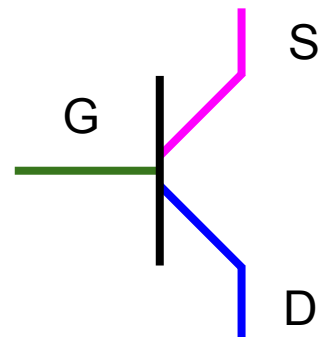
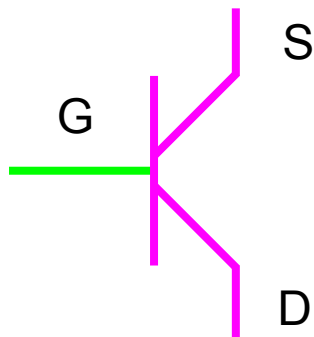


Transistors

Wires transfer information (voltage). Transistors turn voltage into logic.

NMOS Transistor

- When G is high voltage, S and D are electrically connected.
- When G is low voltage, S and D are not connected.

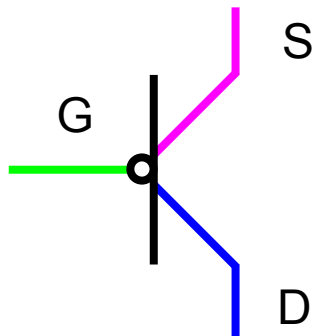
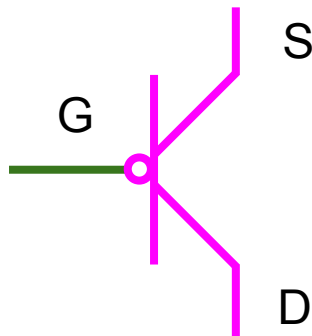


Transistors

Wires transfer information (voltage). Transistors turn voltage into logic.

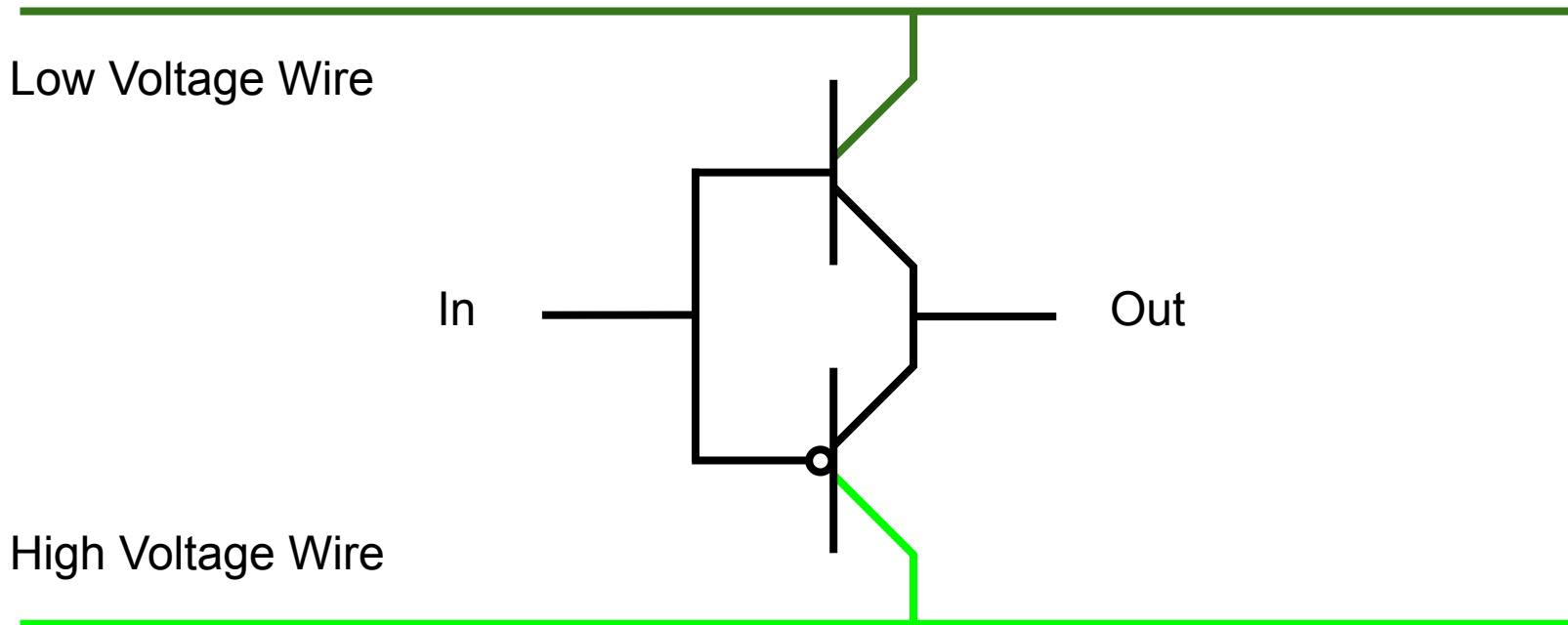
PMOS Transistor

- When G is high voltage, S and D are not connected.
- When G is low voltage, S and D are connected.



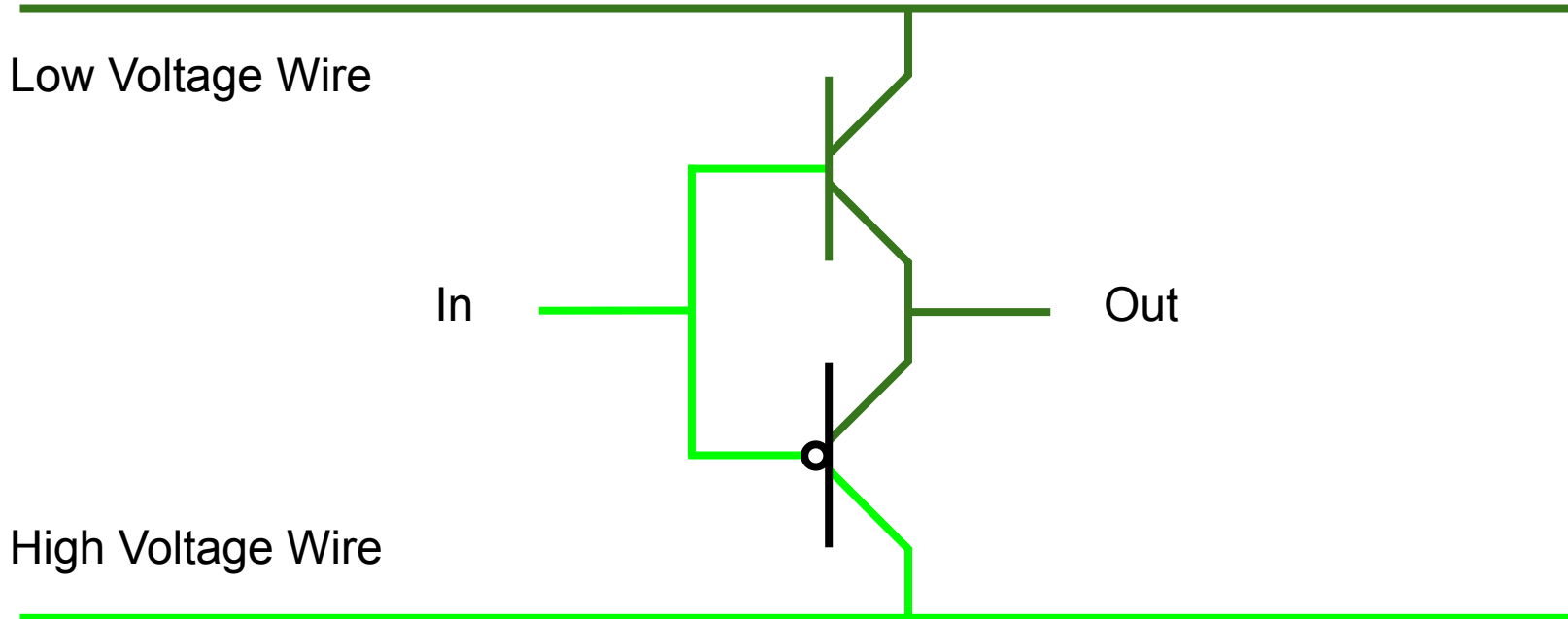
Inverter

Using PMOS + NMOS transistors, we can make logic gates



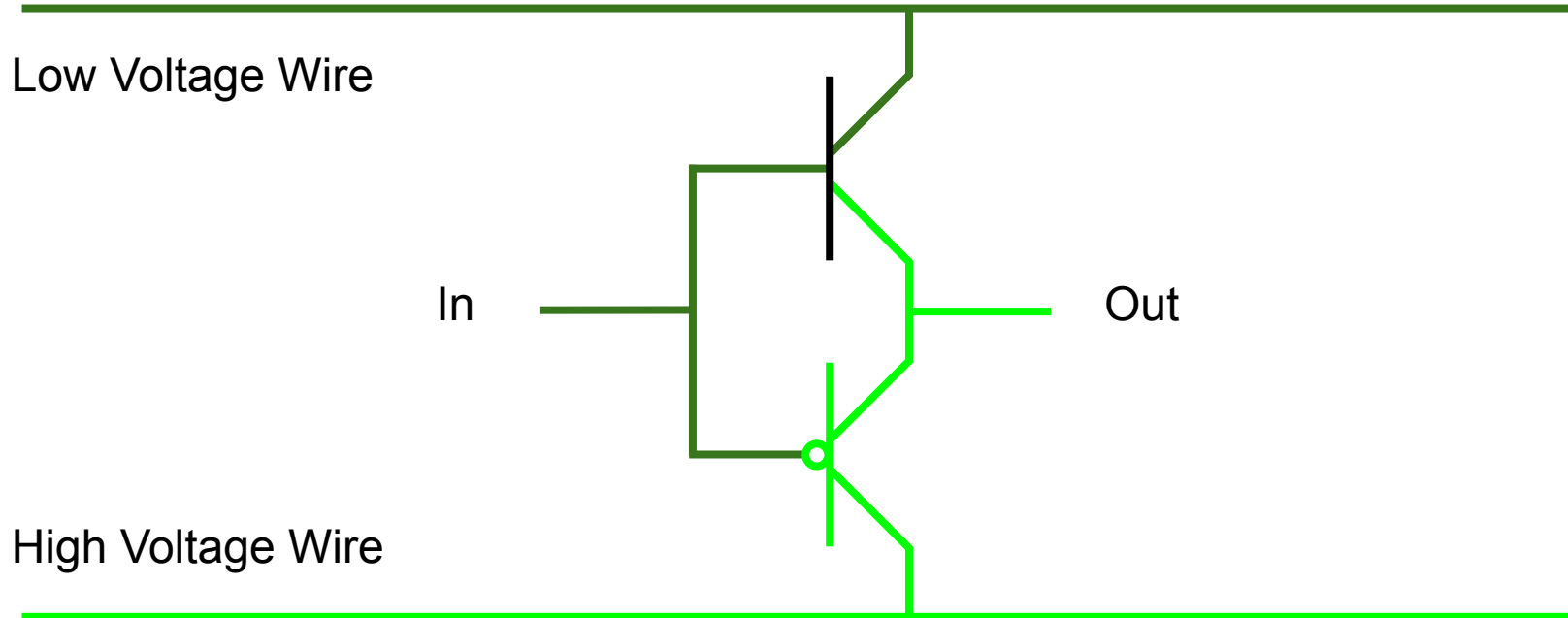
Inverter

When In is 1 (high voltage, TRUE), Out is 0 (low voltage, FALSE)



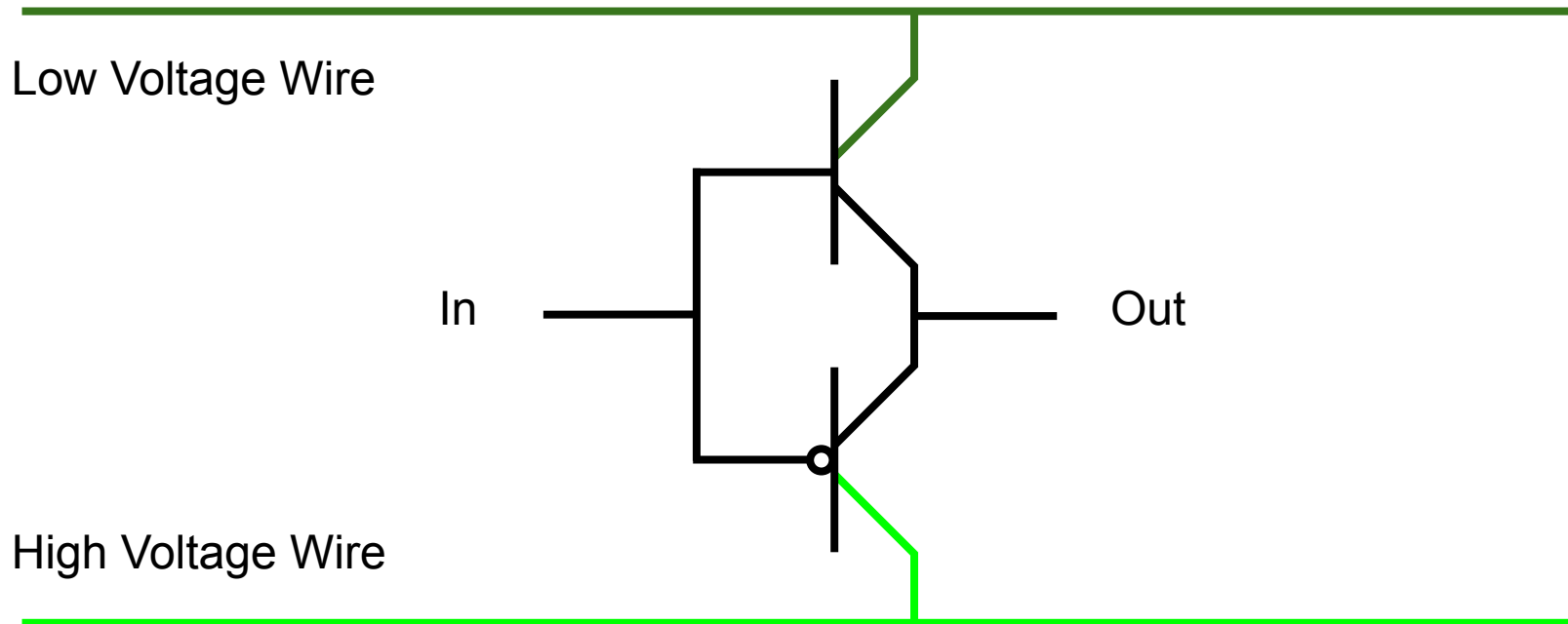
Inverter

When In is 0 (low voltage, FALSE), Out is 1 (high voltage, TRUE)



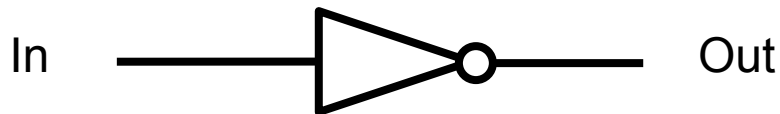
Inverter

$\text{Out} = \neg(\text{In})$. We'll call this a NOT gate (or inverter)

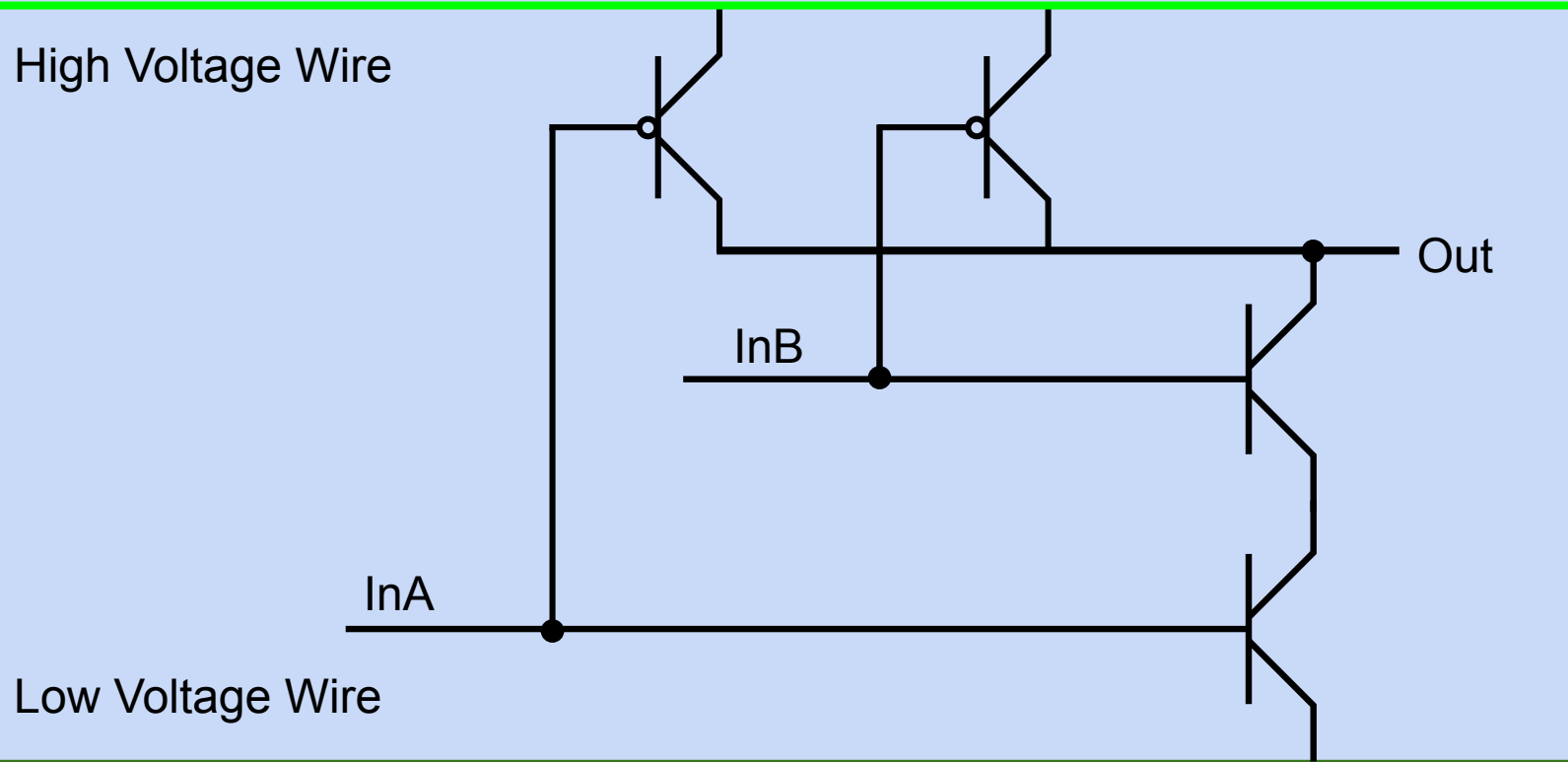


Inverter

$\text{Out} = \neg(\text{In})$. We'll call this a NOT gate (or inverter)

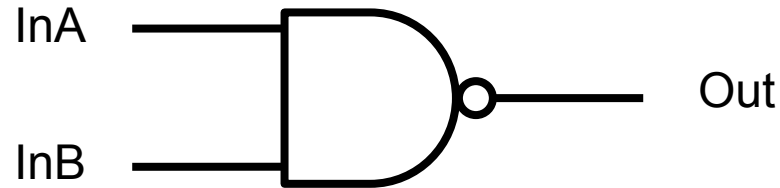


What does this circuit do?



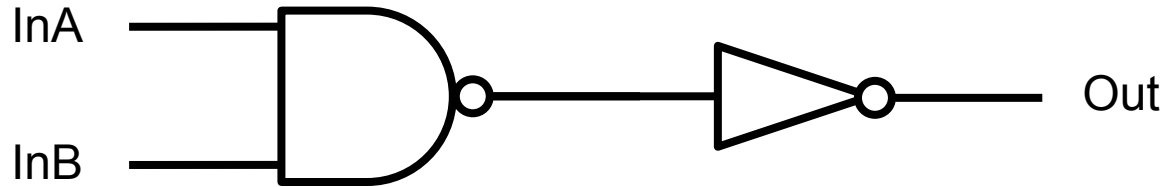
NAND Gate

$\text{Out} = \neg(\text{InA} \ \&\& \ \text{InB})$. We'll call this a NAND gate.

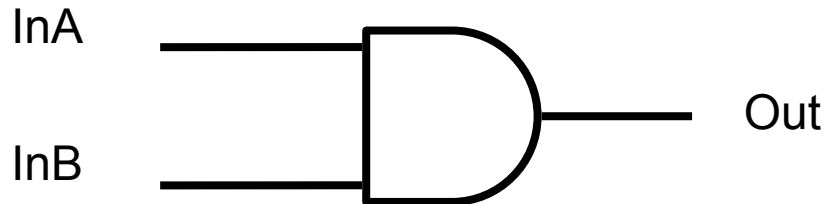
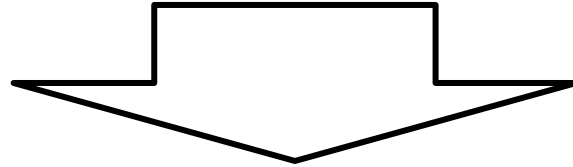


InA	InB	Out
0	0	1
0	1	1
1	0	1
1	1	0

Using NAND and NOT, we can create an AND Gate

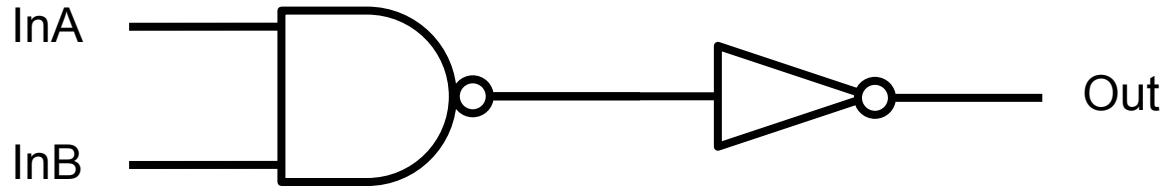


$!(!(A \&\& B)) == A \&\& B$

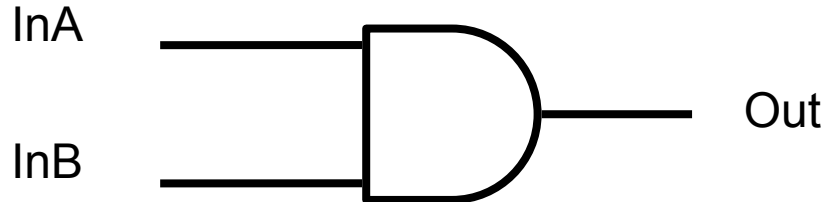
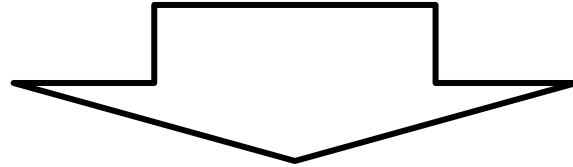


InA	InB	Out
0	0	0
0	1	0
1	0	0
1	1	1

Using NAND and NOT, we can create an AND Gate

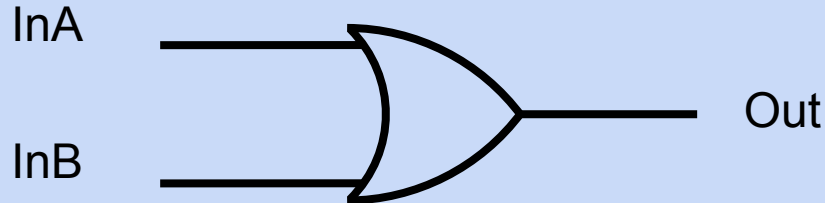
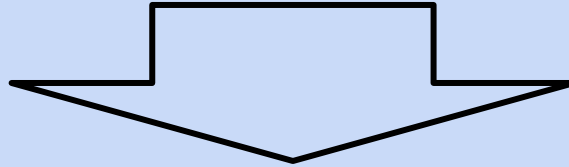
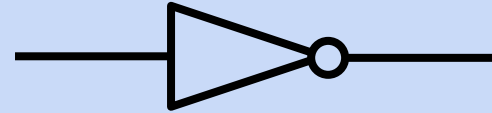
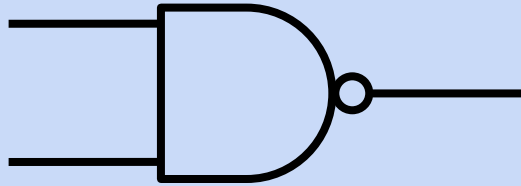


$\neg(\neg(A \ \&\& \ B)) == A \ \&\& \ B$



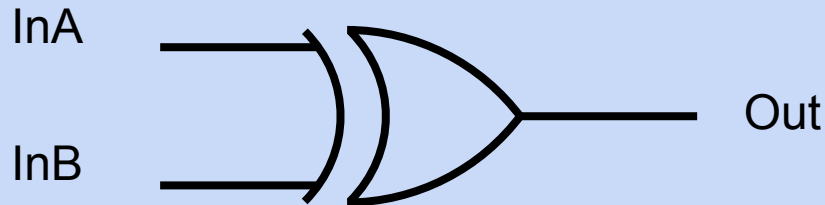
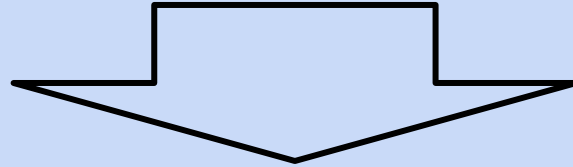
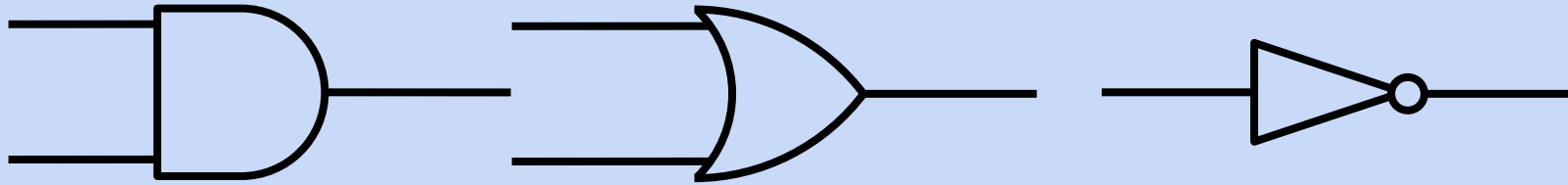
InA	InB	Out
0	0	0
0	1	0
1	0	0
1	1	1

Using NAND and NOT, create an OR Gate



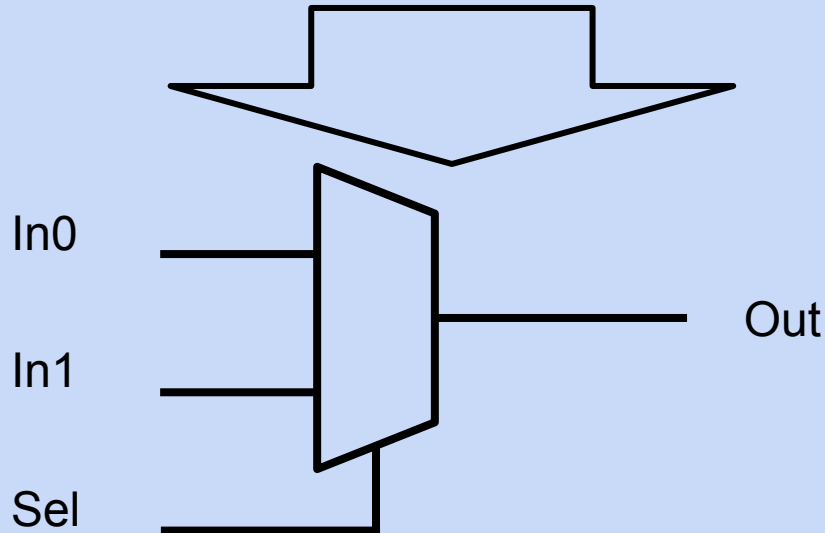
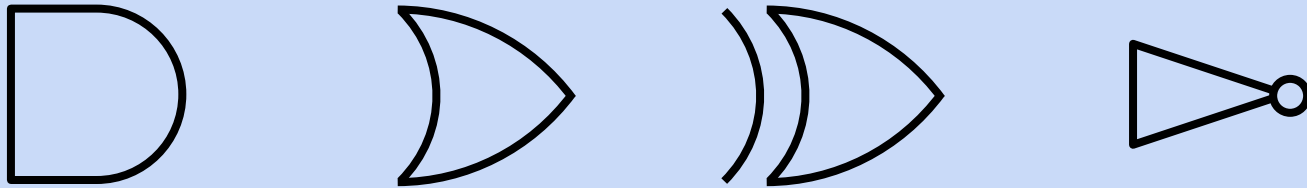
InA	InB	Out
0	0	0
0	1	1
1	0	1
1	1	1

Using AND, NOT, and OR, create an XOR gate



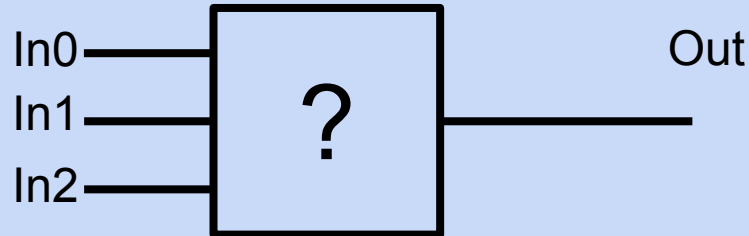
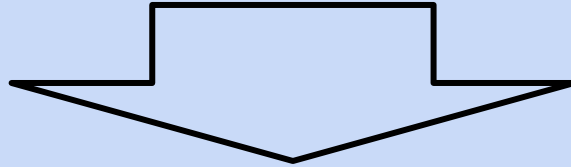
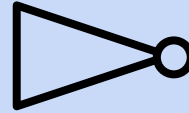
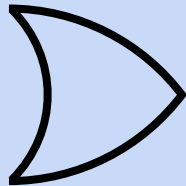
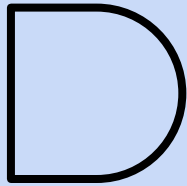
InA	InB	Out
0	0	0
0	1	1
1	0	1
1	1	0

Using AND, NOT, OR, and XOR, create a MUX gate



Sel	Out
0	In0
1	In1

Using AND, NOT, and OR, create an arbitrary circuit



In0	In1	In2	Out
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Scratch Space

Sum of Products

Using just OR, AND, and NOT, we can create a circuit for any given **truth table**.

In the table to the right:

Row 1 = (not In0) and (not In1) and (In2)

Row 2 = (not In0) and (In1) and (not In2)

Row 3 = (not In0) and (In1) and (In2)

Row 5 = (In0) and (not In1) and (In2)

Row 6 = (In0) and (In1) and (not In2)

Out = Row1 or Row2 or Row3 or Row5 or Row6

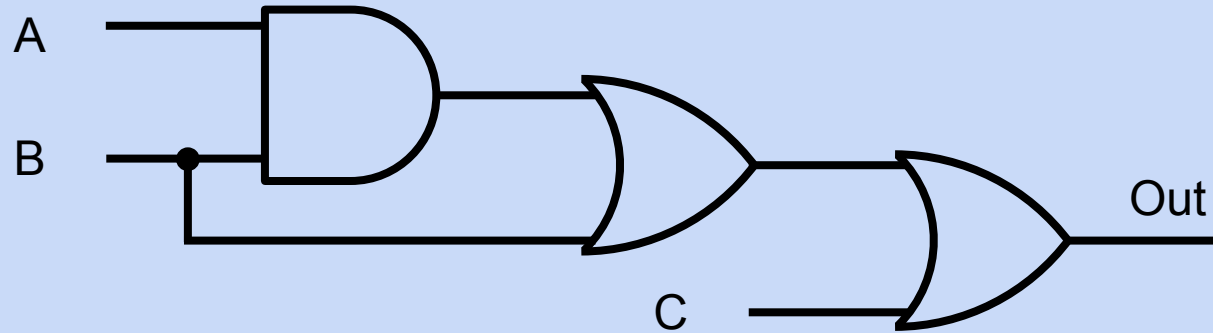
For an arbitrary truth table with N inputs,
Sum-of-Products yields a circuit in at most $O(2^N)$ gates.
Can we do better?

In0	In1	In2	Out
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

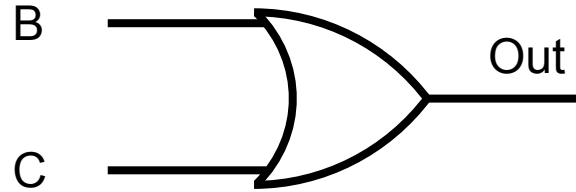
Agenda

- Overview
- Transistors
 - Building basic logic gates
 - Building Circuits
- **Simplifying Circuits**
 - Boolean Logic

Can we make an equivalent circuit with fewer gates?



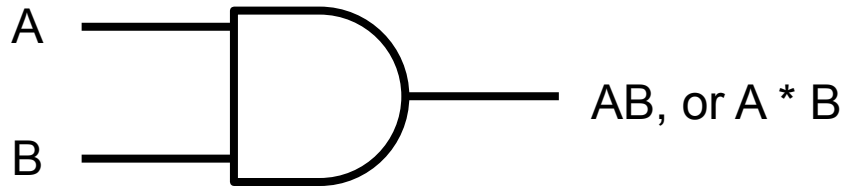
Can we make an equivalent circuit with fewer gates?



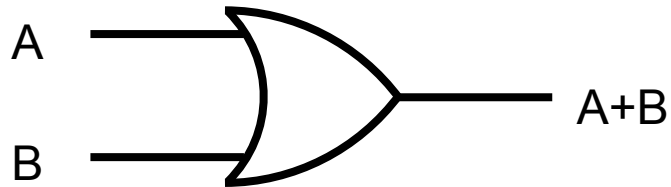
Boolean Algebra

- Every function from $\{0,1\}^N \rightarrow \{0,1\}$ has a corresponding truth table
- Truth tables often have multiple circuits and logical expressions that compute their function
- Need standardized way to notate boolean expressions.
- Boolean Algebra: Use math notation to write basic logic operators (OR, AND, and NOT)

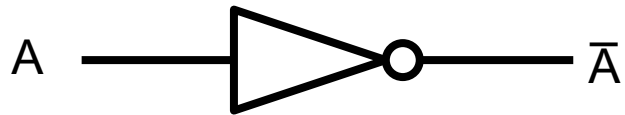
Boolean Algebra



AND: Notated as multiplication



OR: Notated as addition



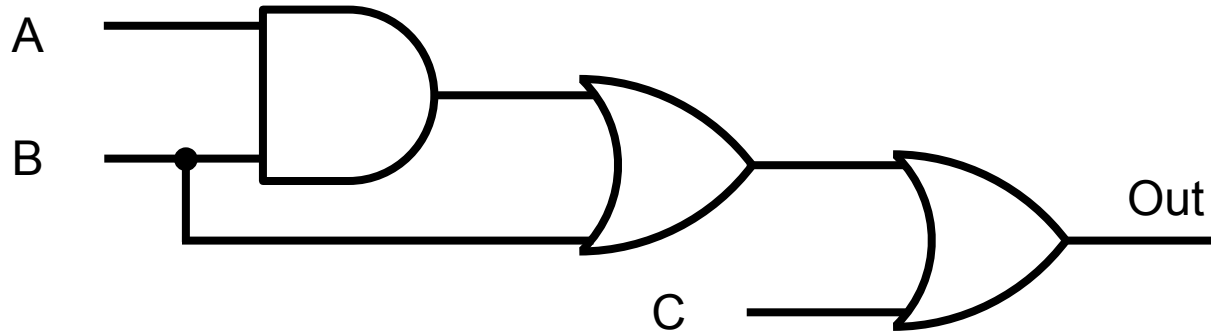
NOT: Notated with overline

1 = TRUE, 0 = FALSE

Boolean Simplification Rules

AND Form	OR Form	
$x \cdot y = y \cdot x$	$x + y = y + x$	Commutativity
$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	Associativity
$x \cdot 1 = x$	$x + 0 = x$	Identity
$x \cdot 0 = 0$	$x + 1 = 1$	Laws of 0's and 1's
$xy + x = x$	$(x + y)x = x$	Uniting Theorem
$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
$x \cdot x = x$	$x + x = x$	Idempotence
$x \cdot \bar{x} = 0$	$x + \bar{x} = 1$	Inverse (Complement)
$\overline{(xy)} = \bar{x} + \bar{y}$	$\overline{(x + y)} = \bar{x} \cdot \bar{y}$	DeMorgan's Laws

Can we make an equivalent circuit with fewer gates?



Equivalent Boolean Expression:

$$((AB)+B)+C$$

$$= B+C$$

Simplifying Sum of Products

$\bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C}$ (21 gates)

$\bar{A}\bar{B}C + \bar{A}B(\bar{C} + C) + A\bar{B}C + AB\bar{C}$

$\bar{A}\bar{B}C + \bar{A}B(1) + A\bar{B}C + AB\bar{C}$

$\bar{A}\bar{B}C + \bar{A}B + A\bar{B}C + AB\bar{C}$

$\bar{A}(\bar{B}C + B) + A\bar{B}C + AB\bar{C}$

$\bar{A}(\bar{B}C + B + BC) + A\bar{B}C + AB\bar{C}$

$\bar{A}(B + \bar{B}C + BC) + A\bar{B}C + AB\bar{C}$

$\bar{A}(B + (\bar{B} + B)C) + A\bar{B}C + AB\bar{C}$

$\bar{A}(B + C) + A\bar{B}C + AB\bar{C}$

$\bar{A}(B + C) + A(\bar{B}C + B\bar{C})$ (10 gates)

Can we do better? Yes (with different simplification order)

Best I could find: $(B + C)(!(ABC)) = 5$ gates

A	B	C	Out
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Bad News

- Is it possible to algorithmically determine the fewest gates for a given Boolean Expression?
 - No known algorithm in polynomial time
 - If you do find one, $P = NP$
- Does a randomly generated circuit have a small circuit?
 - No; Sum of Products is asymptotically optimal in expectation

Good News

We're not dealing with random Boolean expressions!

- Mathematical operations have meaning behind them
 - Tends to lead to simple circuits
- The operations we mandate in RV32 base are precisely the ones with simple circuits
- RV32's instruction translations are chosen specifically to simplify the circuit

Often it's possible to find "patterns" in a truth table and get a small circuit that way

In most cases, we don't expect you to get the absolute smallest circuit for a given truth table

Simplify the truth table (Out0 and Out1 equations)

You may use XOR, represented by $\wedge$

But that uses however many gates
you used earlier to make XOR (OR,
AND, NOT count 1 each)

SOP score: 31 total gates

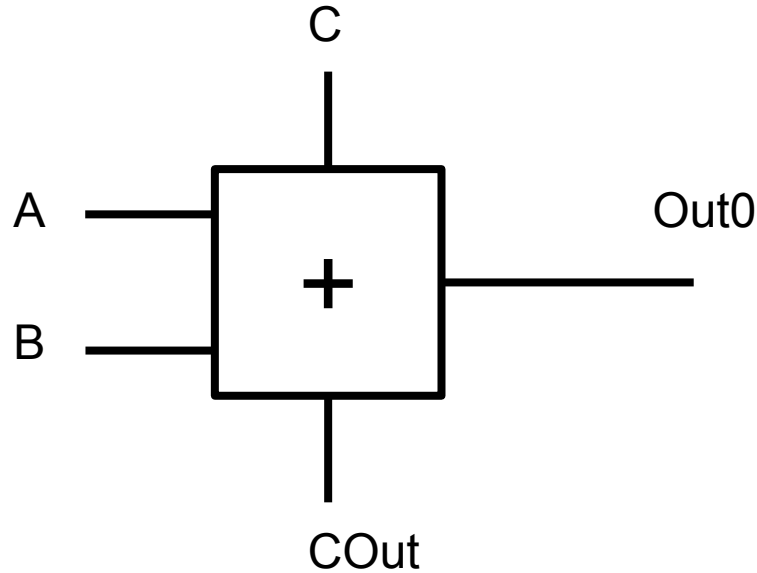
Gold Medal: 15 total gates

Platinum Medal: 12 gates

A	B	C	Out1	Out0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

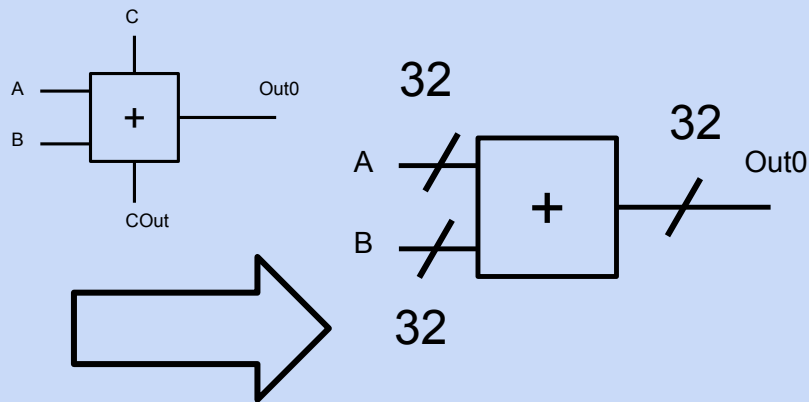
Simplify the truth table (Out0 and Out1 equations)

This circuit is known as a "full adder"
(Adds $A+B+C$)



A	B	CIn	COut	Out0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Create a 32-bit Adder using Full Adder Subcircuits



SOP Score:

~32000000000000000000000000000000 gates

Gold Medal: 500 gates

Platinum Medal: 360 gates

A	B	Out0
0x00000000	0x00000000	0x00000000
0x00000000	0x00000001	0x00000001
0x00000000	0x00000002	0x00000002
0x00000000	0x00000003	0x00000003
...	...	...
0xDEADBEEF	0x12345678	0xF0E21567
...	...	...
0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFFE

Scratch Space